

Using Claude Code With OpenCode Go

A practical backend tutorial for `claude-o-go`

`claude-o-go`

June 18, 2026

Contents

1	Goal	3
2	What You Have	3
3	Run <code>claude-o-go</code> From Anywhere	3
3.1	One-Time Setup	3
3.2	Why This Works	4
3.3	Optional Shell Profile Configuration	4
3.4	Daily Usage	4
4	Why a Proxy Is Needed	4
5	Theory Excursion: Localhost, Ports, and Processes	5
6	Theory Excursion: HTTP Proxy vs Backend	5
7	Model Compatibility	6
8	Request Flow	6
9	Theory Excursion: Streaming, SSE, and Backpressure	7
10	Theory Excursion: Common Network Errors	7
11	Important Files	8
11.1	<code>claude-o-go</code>	8
11.2	<code>opencode-go-claude-proxy.mjs</code>	8

12 Local Checks

9

13 Verification

9

14 Common Problems

9

14.1 Proxy Logs Show Up in Claude Code’s TUI

9

14.2 There’s an issue with the selected model

10

14.3 Double /v1/v1/messages

10

14.4 Claude Code Shows sonnet

10

15 Mental Model

10

1 Goal

This tutorial explains the setup in this repository and why it works. The goal is to keep the Claude Code CLI experience while sending model requests to OpenCode Go instead of Anthropic's Claude API.

Claude Code remains the harness: terminal UI, tool calling, file edits, permissions, prompt handling, and the agent loop. OpenCode Go is the backend that receives the actual model request.

2 What You Have

This directory contains:

- `.env`: your OpenCode Go API key.
- `claude-o-go`: the launcher script you run instead of `claude`.
- `opencode-go-claude-proxy.mjs`: the local proxy between Claude Code and OpenCode Go.
- `opencode-go-claude-example.mjs`: a direct API example that calls OpenCode Go without Claude Code.

The normal command is:

```
cd /home/user/Desktop/opencode-go-w-claude
./claude-o-go
```

For a one-shot prompt:

```
./claude-o-go -p "Reply with exactly: OK"
```

3 Run claude-o-go From Anywhere

You do not need to change into this repository every time, and you do not need to copy it into each new project. Install it once and use `claude-o-go` from any project directory.

3.1 One-Time Setup

Make the launcher executable if needed:

```
chmod +x /home/user/Desktop/opencode-go-w-claude/claude-o-go
```

Create a symlink in a directory already on your PATH:

```
mkdir -p ~/.local/bin
ln -s /home/user/Desktop/opencode-go-w-claude/claude-o-go \
    ~/.local/bin/claude-o-go
```

Make sure `~/.local/bin` is on your `PATH`:

```
export PATH="$HOME/.local/bin:$PATH"
```

Then reload your shell:

```
source ~/.bashrc    # or: source ~/.zshrc
```

3.2 Why This Works

The launcher resolves its own symlink with `readlink -f`. That lets it find the real install directory, where `.env` and `opencode-go-claude-proxy.mjs` live. Claude Code itself starts back in the directory where you ran `claude-o-go`, so the current project remains the working directory.

3.3 Optional Shell Profile Configuration

If you would rather not keep the key in `.env`, export it once in your shell profile:

```
export OPENCODE_GO_API_KEY="sk-..."
export OPENCODE_GO_MODEL="qwen3.7-plus"
```

The launcher picks up `OPENCODE_GO_API_KEY` from the environment and skips reading `.env`. The model is no longer hardcoded in the launcher; define `OPENCODE_GO_MODEL` in your shell profile so it applies everywhere.

3.4 Daily Usage

From any project directory:

```
cd /path/to/any/project
claude-o-go
```

For a one-shot:

```
claude-o-go -p "Explain this repository in one paragraph."
```

Switch models for a single invocation:

```
OPENCODE_GO_MODEL=minimax-m3 claude-o-go
```

4 Why a Proxy Is Needed

OpenCode Go exposes several models through an Anthropic-compatible Messages API. It is a routing and billing layer, not a model host. Each request is routed to the model vendor's own API.

Examples:

```
qwen3.7-plus
minimax-m3
qwen3.7-max
```

Claude Code validates model names locally before sending requests. If you try to call Claude Code directly with an OpenCode Go model, Claude Code rejects it before the request is useful:

```
ANTHROPIC_BASE_URL=https://opencode.ai/zen/go/v1 \
ANTHROPIC_API_KEY=... \
claude --model qwen3.7-plus
```

The proxy solves this by letting Claude Code think it is using a normal Claude alias such as `sonnet`, while the proxy rewrites the outgoing request to use the OpenCode Go model.

5 Theory Excursion: Localhost, Ports, and Processes

Networking starts with a simple address: where should a client connect?

- 127.0.0.1 means localhost. It points back to the same machine.
- A port selects one listening process on that machine. The default proxy port is 4141.
- A server binds to an address and port. The proxy binds to 127.0.0.1:4141.
- A client connects to that address and port. Claude Code connects to the local proxy URL:

```
http://127.0.0.1:4141
```

```
Claude Code process
  connects to 127.0.0.1:4141
```

```
Node proxy process
  listens on 127.0.0.1:4141
  connects outward to opencode.ai:443
```

The proxy listens on localhost, not on 0.0.0.0. That matters: 127.0.0.1 is reachable only from your own machine, while 0.0.0.0 would expose the listener on all network interfaces.

Useful admin checks:

```
ss -ltnp | grep 4141
lsof -iTCP:4141 -sTCP:LISTEN
```

6 Theory Excursion: HTTP Proxy vs Backend

Claude Code is the client. OpenCode Go is the upstream backend. The local Node script is a proxy.

A proxy receives a request from one side and sends a related request to another side. It can preserve most of the request, rewrite selected fields, and relay the response.

In this project the proxy does three important things:

1. It accepts Claude Code's Anthropic-style request at `/v1/messages`.
2. It rewrites `body.model` from Claude's local alias to the OpenCode Go model.
3. It forwards the request to OpenCode Go and streams the response back.

The proxy is not a model host. It does not generate tokens. It is glue code between two APIs that are almost, but not perfectly, compatible.

7 Model Compatibility

Each upstream provider implements its own Anthropic-compatible endpoint, so compatibility varies. Claude Code sends a full Anthropic-style payload, including tools, cache control, tool choice, and Anthropic beta headers.

Model id	Provider	Works with Claude Code harness?
qwen3.7-plus	Alibaba (Qwen)	Yes
minimax-m3	MiniMax	Yes
glm-5.2	Z.ai (Zhipu AI)	No; invalid API parameter
glm-5.1	Z.ai (Zhipu AI)	No; same as glm-5.2
kimi-k2.7-code	Moonshot AI	No; rejects tool function name format
deepseek-v4-pro	DeepSeek	No; expects OpenAI tool shape

Models from the same vendor generally behave like the tested one. Unverified models should be tested before relying on them.

Practical recommendation:

```
export OPENCODE_GO_MODEL="qwen3.7-plus"    # or minimax-m3
```

8 Request Flow

When you run:

```
./claude-o-go -p "Reply with exactly: OK"
```

the flow is:

1. `claude-o-go` reads the OpenCode Go API key from `.env`.
2. It starts the local proxy at `http://127.0.0.1:4141`.

3. It sets `ANTHROPIC_BASE_URL=http://127.0.0.1:4141`.
4. It starts Claude Code with `claude -bare -model sonnet`.
5. Claude Code sends a request to `/v1/messages`.
6. The proxy changes the request body model to the OpenCode Go model.
7. The proxy forwards the request to `https://opencode.ai/zen/go/v1/messages`.
8. OpenCode Go returns the model response.
9. The proxy streams the response back to Claude Code.

9 Theory Excursion: Streaming, SSE, and Backpressure

LLM responses are usually streamed. The backend does not wait until the whole answer is complete; it sends small chunks as they become available.

The common web pattern for this is SSE: Server-Sent Events. An SSE stream is still HTTP, but the response body stays open and sends events over time:

```
data: {"type":"content_block_delta","text":"hello"}
```

```
data: {"type":"content_block_delta","text":" world"}
```

```
data: [DONE]
```

Backpressure means the receiver cannot always consume data as fast as the sender writes it. In Node, `response.write(chunk)` returns `false` when the outgoing buffer is full. Correct proxy code waits for `drain` before continuing. That prevents memory growth and unstable long-running streams.

In this project:

- Claude Code reads from the local proxy.
- The proxy reads from OpenCode Go.
- The proxy writes chunks to Claude Code while respecting Node stream backpressure.
- If Claude Code closes the local connection, the proxy aborts the upstream request.

Stream handling is more delicate than a simple JSON request. A normal request has one response body. A stream is an ongoing conversation between sockets.

10 Theory Excursion: Common Network Errors

These error names are useful admin vocabulary:

- `ECONNREFUSED`: nothing accepted the connection. Typical causes: wrong port, proxy not running, or upstream unreachable.

- **ECONNRESET**: the other side reset an existing connection. Typical cause: the upstream closed the socket mid-stream.
- **UND_ERR_SOCKET**: Node's Undici HTTP client saw a socket failure. Usually the same class as a reset or broken stream.
- **ETIMEDOUT**: connection or response took too long. Typical cause: network path or backend too slow.
- **AbortError**: we intentionally aborted the request. Typical cause: Claude Code closed the local connection.

The practical rule: retry before response headers are sent; after streaming has started, finish the stream cleanly with an error event. Once headers and partial content have been sent, the proxy cannot turn the response into a normal JSON 502.

11 Important Files

11.1 `claude-o-go`

The launcher exports the API key, model, and local base URL, then starts Claude Code:

```
export OPENCODE_GO_API_KEY="$api_key"
export OPENCODE_GO_MODEL="$model"
export ANTHROPIC_BASE_URL="http://127.0.0.1:${proxy_port}"
claude --bare --model "$claude_model_alias" "$@"
```

Important defaults and environment variables:

- **OPENCODE_GO_MODEL**: required; no hardcoded launcher default.
- **OPENCODE_GO_PROXY_PORT**=4141.
- **OPENCODE_GO_UPSTREAM_RETRIES**=2.
- **CLAUDE_CODE_MODEL_ALIAS**=sonnet.
- **CLAUDE_CODE_OPENCODE_GO_BARE**=1.

11.2 `opencode-go-claude-proxy.mjs`

The proxy's core rewrite is:

```
body.model = model
```

For generation requests, the proxy streams OpenCode Go's response back to Claude Code. It handles Node stream backpressure, aborts the upstream request if Claude Code closes the local connection, retries retryable upstream connection failures before response headers are sent, and turns mid-stream upstream resets into a clean streaming error event instead of crashing the proxy process.

The proxy also implements endpoints Claude Code probes:


```
HEAD /
HEAD /v1
GET /v1/models
POST /v1/messages/count_tokens
POST /v1/messages
```

12 Local Checks

Before pushing changes, run:

```
npm run check
```

That syntax-checks both JavaScript entry points.

13 Verification

Run:

```
OPENCODE_GO_PROXY_LOG=1 ./claude-o-go -p "Reply with exactly: OK"
```

Proxy request logs are hidden during normal use so Claude Code's TUI stays clean. With `OPENCODE_GO_PROXY_LOG=1`, look for lines like:

```
OpenCode Go Claude proxy listening on http://127.0.0.1:4141/v1 -> qwen3.7-plus
POST /v1/messages
POST /messages -> 200 as qwen3.7-plus
OK
```

The key line is:

```
POST /messages -> 200 as qwen3.7-plus
```

That means Claude Code sent a request to the local proxy, the proxy forwarded it to OpenCode Go, OpenCode Go returned HTTP 200, and the upstream model was `qwen3.7-plus`.

14 Common Problems

14.1 Proxy Logs Show Up in Claude Code's TUI

Routine request logs are hidden by default. For debugging:

```
OPENCODE_GO_PROXY_LOG=1 claude-o-go
```

In normal use, leave that unset so the TUI stays clean.

14.2 There's an issue with the selected model

Use the launcher:

```
./claude-o-go
```

Do not call `claude -model qwen3.7-plus` directly.

14.3 Double `/v1/v1/messages`

`ANTHROPIC_BASE_URL` should point to the proxy root:

```
http://127.0.0.1:4141
```

It should not include `/v1`, because Claude Code appends `/v1/messages` itself.

14.4 Claude Code Shows `sonnet`

That is expected. Claude Code sees `sonnet`; OpenCode Go receives `qwen3.7-plus`. The proxy log is the source of truth.

15 Mental Model

Claude Code CLI

```
  thinks model = sonnet
  sends Anthropic request
```

|

v

Local proxy

```
  rewrites model = qwen3.7-plus
  forwards request
```

|

v

OpenCode Go

```
  runs the actual model
  returns response
```

|

v

Claude Code CLI

```
  displays the answer
```

The Claude Code user experience stays the same, but the model backend is OpenCode Go.